GUIA RÁPIDO 2: PROGRAMAÇÃO EM QISKIT E ALGORITMOS QUÂNTICOS

Material de Apoio da Palestra Prática (2h) | Palestrante: **Yohanna Dvorak** (ydvorak@id.uff.br)

1. Comandos Essenciais do Qiskit (v2.x)

Comando no Qiskit	O que faz no Circuito?
`qc = QuantumCircuit(n, m)`	Cria n qubits ($\|0\rangle$) e m bits clássicos
`qc.h(0)`	Porta **Hadamard** H em q_0 ($\|0\rangle \to \|+\rangle$)
`qc.x(0)`, `qc.z(0)`, `qc.y(0)`	Portas de **Pauli** X, Z e Y em q_0
`qc.s(0)`, `qc.t(0)`	Portas de **Fase** S (90°) e T (45°)
`qc.ry(theta, 0)`	Rotação contínua $R_y(\theta)$ em q_0
`qc.cx(c, t)`	Porta **CNOT** (controle c, alvo t)
`qc.cz(c, t)`	Porta **Controlled-Z** (fase $-$ em $\|11\rangle$)
`qc.ccx(c1, c2, t)`	Porta de **Toffoli** (2 controles, 1 alvo)
`UnitaryGate(M)`	Cria **porta unitária** customizada U
`porta.inverse()`	Inverte porta/circuito ($U^\dagger = U^{-1}$)
`CDKMRippleCarryAdder(n)`	**Somador de Cuccaro** ($2n+1$ qubits)
`qc.measure([0,1], [0,1])`	Mede qubits q_0, q_1 nos bits c_0, c_1
`qc.draw(output="mpl")`	Desenha o circuito com Matplotlib

2. Simular, Criar e Inverter uma Porta no Qiskit

```
from qiskit import QuantumCircuit
from qiskit_aer import AerSimulator
from qiskit.quantum_info import Statevector
from qiskit.circuit.library import UnitaryGate
import numpy as np

qc = QuantumCircuit(2, 2)
qc.h(0); qc.cx(0, 1)  # Cria par de Bell |Phi+>
sv = Statevector.from_instruction(qc)  # 1. Vetor de estado

# 2. Criar porta unitaria nova U e sua inversa (U^dagger):
M = 0.5 * np.array([[1+1j, 1-1j], [1-1j, 1+1j]])
porta_u = UnitaryGate(M, label="SX")
qc.append(porta_u, [0])              # Aplica U em q0
qc.append(porta_u.inverse(), [0])  # Aplica U^dagger (
    desfaz U!)

# 3. Medir e rodar no simulador com 1024 shots:
qc.measure([0, 1], [0, 1])
counts = AerSimulator().run(qc, shots=1024).result().
    get_counts()
```

3. Perguntas Frequentes (FAQ de Programação)

- **1. Por que aplicar X em q_0 imprime '01' e não '10'?** Porque o Qiskit usa a ordem **Little-Endian**: $|q_{n-1} \dots q_1 q_0\rangle$. O qubit q_0 fica sempre **mais à direita** na string binária!
- **2. Como inverto uma porta ou circuito no Qiskit?** Use `.inverse()` (ex.: `porta.inverse()`, `somador.inverse()` ou `qc.inverse()`). Para portas padrão também existem métodos diretos para $S^\dagger$ e $T^\dagger$: `qc.sdg(0)` e `qc.tdg(0)`.
- **3. Como faço uma porta CCZ (Controlled-Controlled-Z) no Qiskit?** Como $Z = HXH$, basta aplicar Hadamard no alvo antes e depois da porta de Toffoli (`ccx`): `qc.h(2); qc.ccx(0, 1, 2); qc.h(2)`.

4. Resumo dos Exercícios do Jupyter Notebook

Exercício 1: Codificação Superdensa (*Superdense Coding*)

- **Objetivo:** Enviar **2 bits clássicos** ($b_1 b_0$) transmitindo apenas **1 qubit** (q_0).
- **Receita:** 1) Par de Bell (`qc.h(0); qc.cx(0, 1)`). 2) Alice codifica em q_0 (`00`→nada, `01`→`z(0)`, `10`→`x(0)`, `11`→`x(0); z(0)`). 3) Bob decodifica (`qc.cx(0, 1); qc.h(0)`) e mede $|b_1 b_0\rangle$ com 100%!

Exercício 2: Teletransporte Quântico (*Quantum Teleportation*)

- **Objetivo:** Transferir $|\psi\rangle$ de Alice (q_0) para Bob (q_2) usando 1 par de Bell (q_1, q_2).
- **Receita:** 1) Par de Bell entre q_1, q_2 (`h(1); cx(1,2)`). 2) Alice emaranha q_0, q_1 (`cx(0,1); h(0)`). 3) Bob corrige q_2: aplica **Porta** X se $q_1 = 1$ (`cx(1,2)`) e **Porta** Z se $q_0 = 1$ (`cz(0,2)`).

Exercício 3: Busca Quântica de Grover (2 Qubits)

- **Objetivo:** Encontrar um estado marcado $|\omega\rangle$ entre $N = 2^n$ estados em $\mathcal{O}(\sqrt{\mathbf{N}})$ passos.
- **Os 3 Blocos do Circuito:**
  1. **Superposição Uniforme ($|s\rangle$):** `qc.h([0, 1])`.
  2. **Oráculo de Fase (U_ω):** Inverte o sinal de $|\omega\rangle$ ($|\omega\rangle \to -|\omega\rangle$). Para marcar $|11\rangle$: `qc.cz(0, 1)`.
  3. **Difusor (U_s):** $H^{\otimes 2} \to X^{\otimes 2} \to CZ \to X^{\otimes 2} \to H^{\otimes 2}$ nos qubits `[0, 1]`. Mede $|11\rangle$ com **100%** em 1 passo!

Exercício 4: Porta `CDKMRippleCarryAdder` + Grover ($A+Y=X$)

- **Como funciona a porta (`kind="fixed"`):** É o **Somador de Cuccaro** reversível de n bits. Usa $\mathbf{2n+1}$ qubits:

$$\underbrace{|A\rangle}_{n \text{ qubits}} \underbrace{|B\rangle}_{n \text{ qubits}} \underbrace{|0\rangle}_{1 \text{ aux}} \xrightarrow{\text{somador}} |A\rangle\,|\mathbf{A+B} \bmod \mathbf{2^n}\rangle\,|0\rangle$$

  Para $n = 2$ (5 qubits): `q0,q1` = reg. A (incógnita); `q2,q3` = reg. B ($Y \to A+Y$); `q4` = aux (`help`). Para $n = 3$ (7 qubits, na demonstração): `q0..q2` = A; `q3..q5` = B; `q6` = aux.
- **Receita para resolver $A + Y = X$ com Grover:**
  1. Inicialize Y em B (portas X) e coloque A em superposição: `qc.h(qubits_A)`.
  2. **Oráculo:** 1) **Soma:** aplica `somador`; 2) **Marca X:** aplica CZ (em $n = 2$) ou CCZ (em $n = 3$) no registrador B; 3) **Descomputa:** aplica `somador.inverse()` para desfazer a soma e desemaranhar B de A!
  3. Aplique o **Difusor de Grover** apenas no registrador A e meça A!

Como rodar: Abra `pratica/pratica_qiskit_aluno.ipynb` no **Google Colab** ou localmente | **Referências:** M. A. Nielsen & I. L. Chuang (*Quantum Computation and Quantum Information*), **IBM Quantum Learning** (`learning.quantum.ibm.com`) e `https://lnkd.in/p/d46rDy9P`